✨ Module 1

12 October 2026 20:27

1. Define Computational Thinking and State its Core Pillars

Computational Thinking:

Computational Thinking is a problem-solving approach that breaks complex problems into manageable steps so that solutions can be implemented efficiently by humans or computers.

Core Pillars:

1. **Decomposition** – Breaking a complex problem into smaller parts.
2. **Pattern Recognition** – Identifying similarities or recurring trends.
3. **Abstraction** – Focusing on important details while ignoring irrelevant ones.
4. **Algorithm Design** – Creating a step-by-step solution to solve the problem.

---

2. What is a Flowchart? Explain Common Flowchart Symbols

Flowchart:

A flowchart is a graphical representation of an algorithm or process using standard symbols connected by arrows.

1. **Terminal (Oval)** – Represents Start or End of a process.
2. **Process (Rectangle)** – Indicates a computation or processing step.
3. **Input/Output (Parallelogram)** – Used for reading input or displaying output.
4. **Decision (Diamond)** – Represents a condition with Yes/No branching.



3. What is an Algorithm? State Any Two Essential Characteristics

Algorithm:

An algorithm is a finite sequence of well-defined steps used to solve a problem or perform a task.

1. **Definiteness** – Every step must be clear and unambiguous.
2. **Finiteness** – The algorithm must terminate after a finite number of steps.
3. **Input** – Accepts zero or more inputs.
4. **Output** – Produces at least one result.

---

4. Explain the Overview and Important Features of C Language

C Programming Language:

C is a general-purpose, procedural programming language developed by Dennis Ritchie in 1972.

1. **Simple and Efficient** – Easy syntax and fast execution.
2. **Portable** – Programs can run on different systems with minor changes.
3. **Structured Language** – Supports modular programming using functions.
4. **Rich Library Support** – Provides many built-in functions and operators.

5. Explain the Basic Structure of a C Program with Example

Basic Structure

1. **Header Files** – Inclusion of required libraries using `#include`.
2. **Main Function** – Execution starts from `main()`.
3. **Declarations** – Variables are declared before use.
4. **Statements & Return** – Logic implementation and program termination.

Example:

C

```c
#include<stdio.h>

int main()
{
    printf("Hello World");
    return 0;
}
```

6. Explain the Role of a Compiler in C Program Execution

1. **Translates Source Code** – Converts C code into machine code.
2. **Error Detection** – Identifies syntax and semantic errors.
3. **Optimization** – Improves program efficiency and performance.
4. **Object Code Generation** – Produces executable files for execution.

T
E
O
O

---

7. Differentiate Between Keywords and User-Defined Identifiers

Keywords	User-Defined Identifiers
Reserved words with predefined meaning.	Names given by the programmer.
Cannot be used as variable names.	Used for variables, functions, arrays, etc.
Fixed and predefined by C language.	Created as per requirement.
Example: `int`, `while`	Example: `total`, `marks`

---

8. What are Variables and Constants in C?

Variable: A named memory location whose value can change during execution.

Constant: A fixed value that cannot be changed during program execution.

1. **Variable Declaration:** `int a;`
2. **Variable Initialization:** `int a = 10;`
3. **Constant Declaration:** `const int MAX = 100;`
4. Variables are mutable, whereas constants remain unchanged.

Example:

C

```c
int age = 18;
const float PI = 3.14;
```

---

9. Explain Different Data Types Available in C

1. **int** – Stores integer values. Example: `int age = 20;`
2. **float** – Stores decimal values. Example: `float price = 99.5;`
3. **char** – Stores a single character. Example: `char grade = 'A';`
4. **double** – Stores large decimal values with higher precision. Example: `double salary = 25000.75;`

10. Explain Arithmetic, Relational, Logical and Assignment Operators

1. **Arithmetic Operators** – Perform mathematical operations.
   Example: `+ , - , * , / , %`
2. **Relational Operators** – Compare values and return true/false.
   Example: `== , != , > , <`
3. **Logical Operators** – Combine conditional expressions.
   Example: `&& , || , !`
4. **Assignment Operators** – Assign values to variables.
   Example: `= , += , -=`

Example:

C

```c
int a = 10, b = 5;
int sum = a + b;      // Arithmetic
a > b;                // Relational
a > 0 && b > 0;       // Logical
a += 5;               // Assignment
```